



Lab 01: Introduction to UNIX & Basic UNIX Commands

What is an Operating System (OS)?

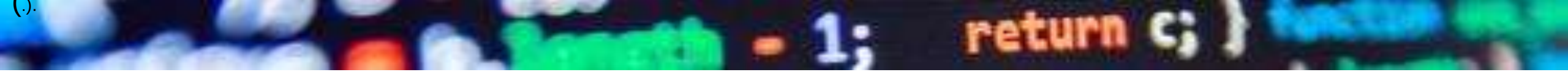
An **Operating System** is system software that:

- Acts as a **bridge between the user and the computer hardware**
- Manages all basic operations of the computer

Without an OS, a computer cannot work.

Types of OS:

Windows, Linux, macOS, Android



Main Tasks Performed by an OS

1. Processor Management

Decides **which program runs on the CPU and when**

2. Memory Management

Allocates and frees **RAM** for programs

3. Device Management

Controls devices like **keyboard, mouse, printer**

4. Application Management

Helps applications run properly

5. Storage Management

Manages files and folders on **hard disk**

6. User Interface

Allows users to interact (GUI or Command Line)



What is UNIX?

UNIX is an operating system that:

- Allows **multiple users**
- Supports **running multiple programs at the same time**
- Is widely used in **servers and programming**

- **Developed in 1969**
- **Created at Bell Labs**

Important Features of UNIX

- **Multiuser** → Many users can use the system at the same time
- **Multitasking** → One user can run many programs simultaneously
- **Portable** → Runs on different hardware
- **Secure** → Strong file permissions



Basic Components of UNIX

1. Kernel

- The **core** of OS
- Communicates with **hardware**
- Manages memory, CPU, files

2. Shell

- Interface between **user and kernel**
- Accepts **commands**
- Converts commands into actions

Examples of shells: Bourne Shell, C Shell, Korn Shell

3. Commands & Utilities

- various commands used to perform tasks, **Examples:**
- ls** (List): Lists the files and directories within a given directory.
- cp** (copy): Copies files or directories from one location to another.
- Rm** (remove): Deletes files or directories
- cat (concatenate)**: Reads, displays, and concatenates (joins) the contents of files.

4. Files & Directories

- Everything in UNIX is **treated as a file**
- Files are stored inside directories**
- Structure looks **like a tree**



Basic UNIX Commands

1. **Date** Displays current date and time
2. **Cal** Displays calendar
3. **echo** Prints text on screen
4. **Ls** Lists files in a directory
5. **Whoami** Shows current logged-in user
6. **Uptime** Shows how long the system is running
7. **uname** Shows system information
8. **Hostname** Displays system name
9. **Bc** Best Calculator in UNIX
10. **Clear** Clears terminal screen
11. **Rm** Deletes a file `rm file1.txt`
12. **Wc** Counts lines, words, characters



Lab 02: Page Replacement Algorithms

What is Page Replacement?

- Main memory (RAM) is **limited**
- Programs are divided into **pages**
- Pages are loaded into **frames**
- If memory is full and a new page is needed → **one existing page must be removed**

The method used to decide **which page to remove** is called a **Page Replacement Algorithm**.



Page Fault

A **page fault** occurs when:

- A page requested by the CPU is **not present in memory**

Each page fault causes:

- Delay in Disk access
- Performance degradation

Common Page Replacement Algorithms

1. FIFO (First In First Out)
2. LRU (Least Recently Used)
3. Optimal
4. MRU (Most Recently Used)



What Is Happening in These Programs?

Each program (FIFO / LRU / Optimal / MRU):

1. Takes a **page reference string** (sequence of page numbers)
2. Takes **number of frames** (memory slots)
3. Simulates **how pages are loaded into memory**
4. Shows:
 1. Which pages are in memory **after each request**
 2. When a **page fault** occurs
5. Finally prints **total page faults**

Common Steps in ALL Programs

No matter which algorithm:

1. Memory frames start **empty**
2. Pages arrive **one by one**
3. For each page:
 1. If page is already in memory → **No page fault**
 2. If page not in memory:
 1. **Page fault occurs**
 2. **If memory full → replace a page (algorithm decides which)**
4. **Display frame contents**
5. **Count page faults**

ALGORITHMS

1. FIFO Program

Logic

- Frames work like a **queue**
- First page that enters memory is **first removed**
- Program remembers **order of arrival**

What Program Does

- Reads reference string
- Inserts pages into frames
- When frames are full:
 - Removes **oldest page**
- Prints memory state after each step

EXAMPLE

Input:

Reference string: 1 2 3 4 1

Frames: 3

Output on screen:

1 - - PF No. 1

1 2 - PF No. 2

1 2 3 PF No. 3

4 2 3 PF No. 4 (1 removed)

4 1 3 PF No. 5

Final:

Total Page Faults = 5

ALGORITHMS

2. LRU Program

Logic

- Replace the **page not used for the longest time**
- Program keeps track of recent usage

What Program Does

- Stores when each page was last used
- **On replacement: Finds least recently used page**
- Replaces it
- Prints frame contents

EXAMPLE

Input:

Reference string: 1 2 3 2 4

Frames: 3

Output on screen:

1 - - PF No. 1

1 2 - PF No. 2

1 2 3 PF No. 3

1 2 3

4 2 3 PF No. 4

Final:

Total Page Faults = 4

ALGORITHMS

3. Optimal Program

Logic

- Replace the page that **will be used farthest in the future**
- Program **looks ahead** in the reference string

What Program Does

- Checks **future references**
- **Finds which page: Is used last or never again**
- **Replaces that page**

EXAMPLE

Input:

Reference string: 2 3 1 4 3 2

Frames: 3

Output on screen:

2 - - PF No. 1

2 3 - PF No. 2

2 3 1 PF No. 3

2 3 4 PF No. 4 (Page 1 will not be used again → REMOVE IT)

Final:

Total Page Faults = 4

ALGORITHMS

4. MRU Program

Logic

- Removes the **most recently used page**
- **Opposite of LRU**

What Program Does

- Tracks **last used page**
- When replacement needed:
 - **Removes most recent one**

EXAMPLE

Input:

Reference string: 1 2 3 2 4

Frames: 3

Output on screen:

1 - - PF No. 1

1 2 - PF No. 2

1 2 3 PF No. 3

1 2 3

1 4 3 PF No. 4

Final:

Total Page Faults = 4



EXERCISE

- **Reference String:** 1 2 3 4 1 2 5 1 2 3 4 5
- **Number of Frames:** 3

Using the above reference string and number of frames, **simulate the following page replacement algorithms:**

- 1.FIFO (First In First Out)
- 2.LRU (Least Recently Used)
- 3.Optimal
- 4.MRU (Most Recently Used)

For **each algorithm:**

- Frame contents after **each page reference**
- Page fault occurrences
- Total number of page faults**



Lab 03: Process Scheduling Algorithms

Scheduling Algorithms

- 1.FCFS (First Come First Serve)
- 2.SJF (Shortest Job First)
- 3.Round Robin
- 4.Priority Scheduling

Terminologies Used in CPU Scheduling

- Arrival Time:** The time at which the process arrives in the ready queue.
- Completion Time:** The time at which the process completes its execution.
- Burst Time:** Time required by a process for CPU execution.
- Turn Around Time:** Time Difference between completion time and arrival time.

$$\text{Turn Around Time} = \text{Completion Time} - \text{Arrival Time}$$

- Waiting Time(W.T):** Time Difference between turn around time and burst time.

$$\text{Waiting Time} = \text{Turn Around Time} - \text{Burst Time}$$

FCFS (First Come First Serve)

Idea

- Processes execute in the **order they arrive**
- Like a **queue at a counter**

How FCFS Works

1. First process runs completely
2. Next process waits until CPU is free
3. Continue until all processes finish

Process	Burst Time
P0	4
P1	3
P2	2

Execution Order: P0 → P1 → P2

Key Points

Simple

Long waiting time

SJF (Shortest Job First)

Idea

- Process with **smallest burst time executes first**
- If same burst time → FCFS

How SJF Works

1. Sort processes by burst time
2. Execute shortest job first
3. Calculate WT and TAT

Process	Burst Time
P0	6
P1	2
P2	4

Execution Order: P1 → P2 → P0

Key Points

Minimum average waiting time
Starvation possible

Round Robin (RR)

Idea

- Each process gets **fixed time quantum**
- CPU switches between processes in **circular order**

Time Quantum = 3

Process	Burst Time
P0	5
P1	4
P2	3

How SJF Works

1. Give CPU to process for time quantum
2. If not finished → move to end of queue
3. Repeat until all finish

Execution Order: **P0 → P1 → P2 → P0 → P1**

Key Points

Fair scheduling
No starvation

Priority Scheduling

Idea

- CPU assigned based on **priority**
- **Lower number = higher priority**
- Same priority → FCFS

How SJF Works

1. Sort processes by priority
2. Execute highest priority first
3. Calculate WT and TAT

Execution Order: P1 → P0 → P2

Key Points

Important tasks run first
Starvation possible

Process	Burst	Priority
P0	4	2
P1	3	1
P2	2	3



Lab 04: Intro To Threads

What is a Process?

A **process** is a program in execution.

Example: When you open a browser, it becomes a process.

What is a Thread?

A **thread** is a small unit of execution inside a process that allows tasks to run concurrently.

Simply, Thread allows a program to do multiple things at once

Example: A music player playing music (**one thread**) while downloading a new song (**another thread**) at the same time.

(.)

Process vs Thread

Process	Thread
Heavy	Lightweight
Own memory	Shared memory
Slower	Faster
Independent	Inside a process

Note: A process can have multiple threads



Why Are Threads Needed?

Threads are used to:

- Perform multiple tasks at the same time
- Improve program performance
- Use CPU efficiently

Example:

- One thread prints data
- Another thread performs calculations



Program Explanation

This lab is **only meant to show**:

- How threads are created
- That thread execution order is **not fixed**
- That multiple threads can run inside one program

What the Program Does?

- 1.The main program starts
- 2.Two threads are created
- 3.Each thread prints a message
- 4.Main program waits for both threads to finish
- 5.Program terminates



Program Explanation

Why Output Order Changes?

Thread scheduling is handled by the operating system.

Therefore:

- Thread 1 may print first
 - Thread 2 may print first
 - Order is **not guaranteed**
-
- **Return value 0 indicates:**
 - **Thread was created successfully.**

Output may appear as:

Thread 1

Thread 2

Thread 1 returns: 0

Thread 2 returns: 0

OR

Thread 2

Thread 1

Thread 1 returns: 0

Thread 2 returns: 0

Both outputs are correct.

Errors when pthread_create() does NOT return 0

Any non-zero return value means the thread could not be created due to system or configuration issues.

Error Type	Means
Insufficient resources	System does not have enough memory or capacity to create a thread
Invalid function	Thread function is not defined or incorrectly specified
Thread library not linked	Program not compiled with -pthread
Environment restriction	OS or execution environment blocks thread creation



Compilation and Execution Steps (Ubuntu)

1. Open terminal
2. Create a file : `nano threads.c`
3. Paste the given code and save
4. Compile using: `gcc -pthread threads.c -o threads`
5. Run the program: `./threads`



Lab 05: Producer Consumer Problem

Cooperating vs Independent Processes

Independent Process

- Shares **NO data**
- Cannot affect others
- Example:
 - Calculator app
 - Notepad app

Cooperating Process

- Shares **data or resources**
- Can affect or be affected by others
- Example:
 - Browser + download manager
 - Editor + compiler

So:

If processes share data → they are cooperating.



Inter-process Communication

IPC is just how different running programs (processes) talk to each other.

Cooperating processes need **interprocess communication (IPC)**

Two models of IPC

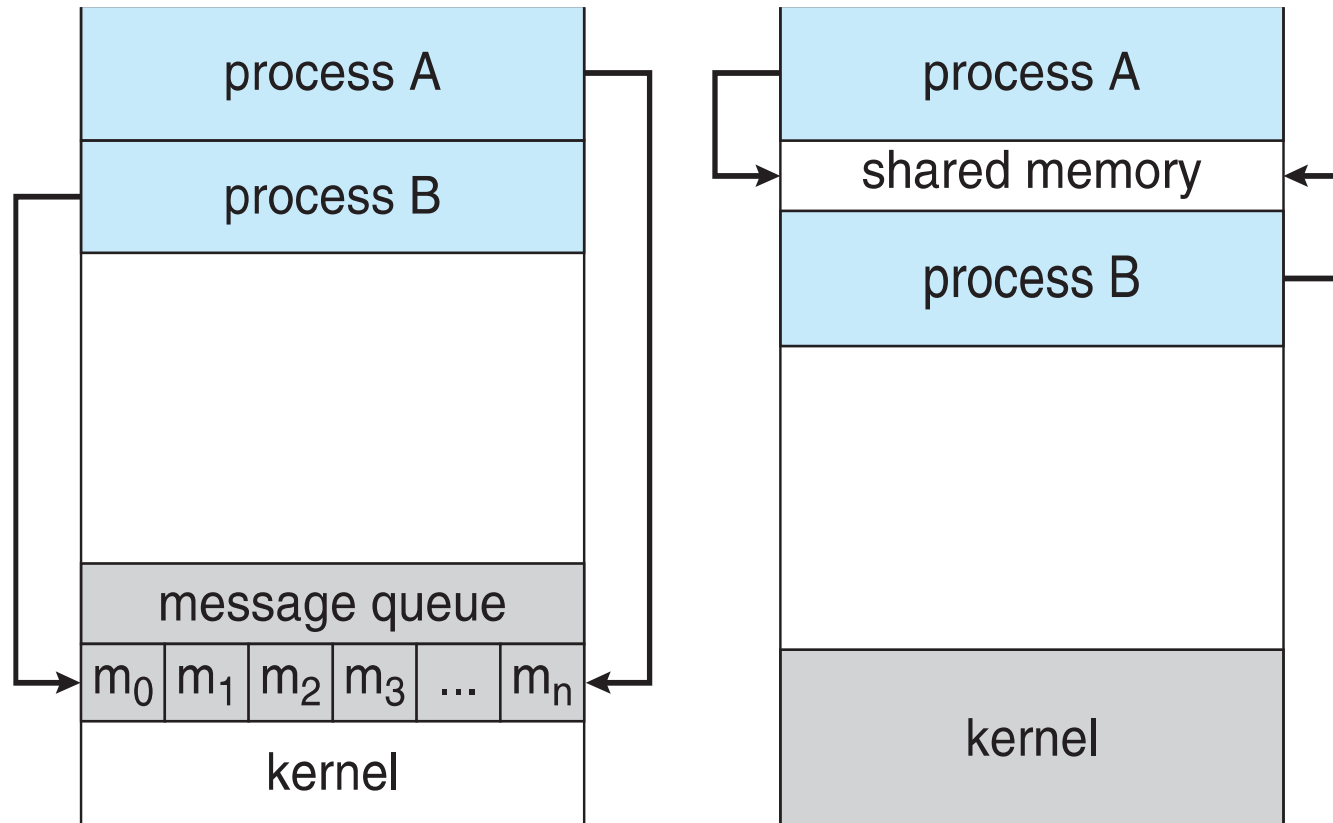
Message passing

Processes send messages to each other through the operating system (kernel).
They do NOT share memory.

Shared memory:

Processes share the same memory space.
They read and write directly to it.

(a) Message passing. (b) shared memory.



- Process A → sends a message
 - Kernel → stores it in a **message queue**
 - Process B → receives the message
- The kernel controls everything.**

- OS creates a **shared memory region**
- Process A and B both attach to it
- Data is **written/read directly**
- **Kernel sets it up once, then steps back.**



Core problem in shared memory

Multiple processes access the same data at the same time, and that can mess things up because multiple processes may access and modify shared data simultaneously, leading to **data inconsistency or data corruption.**

- That's why Shared memory requires **synchronization***

***processes synchronization is coordination mechanism in OS that manages concurrent process execution to ensure data consistency and prevent conflict**



Producer-Consumer problem

The Producer-Consumer problem exists to safely exchange data between two independent activities using a temporary storage area called a buffer.

Key Roles

Producer

- Creates or generates data
- Example: keyboard

Consumer

- Uses or processes data
- Example: text editor reading keys, printer, display

Buffer

- Temporary **waiting area** between producer and consumer
- Stores data until it is used



Example 1: WhatsApp Voice Message

- Recording voice → Producer
- Listening later → Consumer
- Stored audio → Buffer

Example 2: Keyboard Input

- Keyboard sends characters fast
- Program reads characters slower
- OS buffer stores keystrokes

Example 3: Printer Spooling

- Program sends data fast
- Printer prints slowly
- Print queue acts as buffer

Why Buffer Size Matters?

-Small Buffer

- Producer waits often
- Consumer gets data late

-Large Buffer

- More memory usage
- Less waiting

OS designers choose buffer size carefully.

If producer and consumer access buffer without control?

Situation	What Goes Wrong
Produce when buffer is full	Old data overwritten
Consume when buffer is empty	Invalid or garbage data

This is why **synchronization** is required, to ensure

- Producer waits when buffer is full
- Consumer waits when buffer is empty



Program

- Buffer = Temporary storage
- Buffer size = 10
- Implemented using an array

Two pointers are used:

- in → where producer inserts data
- out → where consumer removes data

Waiting Conditions

Buffer Full Condition

No empty space is available in buffer

Buffer Empty

No data available to consume

- Producer waits when buffer is full
- Consumer waits when buffer is empty

What the Program Does?

1. Program starts

2. User selects an option:

1. Produce
2. Consume
3. Exit

3. If **Produce**:

1. Check if buffer is full
2. If not full, insert item

4. If **Consume**:

1. Check if buffer is empty
2. If not empty, remove item

5. Steps repeat until exit



Lab 06: Process Synchronization

When multiple processes or threads run **at the same time and share data**, problems can occur if they access the **shared data simultaneously**. This situation is called a **Critical Section Problem**.

Why is it a problem?

Imagine:

- Two people editing the same Google Doc at the same time.
- Two ATMs withdrawing money from the same bank account simultaneously.

Without control:

- Data may become **corrupted**.
- Results may become **inconsistent**.
- System may **crash**.

So we need **process synchronization** to control access.

Readers–Writers Problem

This is a classic Operating Systems problem.

There is:

- **A shared resource** (like a file, database, memory)
- **Two types of processes:**
 - **Readers**
 - **Writers**

Rules of the Problem

1. **Many readers** can read at the same time.
2. **Only ONE writer** can write at a time.
3. **When a writer is writing:**
 1. No readers allowed.
 2. No other writers allowed.
4. **A writer must wait if even 1 reader is reading.**



It ensures:

- ✓ Data consistency
- ✓ No corruption
- ✓ Proper coordination
- ✓ Safe concurrency



Readers–Writers Problem

Example Scenario

1. Reader1 enters → locks writer access
2. Reader2 enters → allowed
3. Reader3 enters → allowed
4. Writer tries → waits
5. All readers leave
6. Writer enters



Readers–Writers Problem

How Do We Solve It?

We use a synchronization tool called: **Semaphore**

A semaphore (Locking mechanism) - variable used to control access to shared resources.

Variables Used:

1. $x \rightarrow$ protects readercount
2. $y \rightarrow$ protects shared resource
3. **readercount** $\rightarrow$ number of readers inside

What the Program Does?

Program Starts

$x = 1$
 $y = 1$
 $\text{readercount} = 0$

If Reader Comes

Step 1:

Lock x
Increase readercount

Step 2:

If $\text{readercount} == 1$
→ Lock y
→ Block writers

Step 3:

Unlock x

Step 4:

Reader reads

When Reader Leaves

Step 1:

Lock x
Decrease readercount

Step 2:

If $\text{readercount} == 0$
→ Unlock y
→ Allow writers

Step 3:

Unlock x

If Writer Comes

Step 1:

Try to lock y
If $y == 0 \rightarrow \text{WAIT}$
If $y == 1 \rightarrow \text{Lock } y$

Step 2:

Writer writes

Step 3:

Unlock y



Important Observations

- **x becomes 0** only when a reader is updating readercount
- **y becomes 0**:
 - When first reader enters
 - When writer is writing
- Multiple readers can read when $y = 0$
- Writer can write only when $y = 1$ and $\text{readercount} = 0$

Condition Summary

1. Resource Busy Condition

$y == 0$

Means: Someone is using resource

2. Readers Active Condition

$\text{readercount} > 0$

Means: Readers inside & Writer must wait

3. Resource Free Condition

$y == 1 \text{ AND } \text{readercount} == 0$

Means: Anyone can enter (Reader or Writer)

4. Reader Waiting Condition

Reader waits only when:

Writer is writing ($y == 0 \text{ AND } \text{readercount} == 0$)

How It Repeats

Program creates multiple reader and writer threads.

For each:

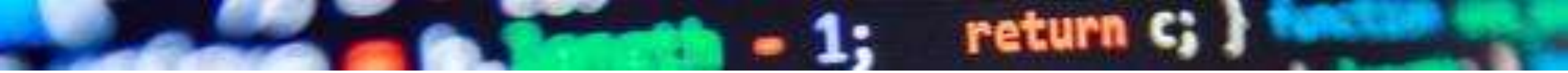
Reader:

- Check, Enter, Read & Exit

Writer:

- Check, Enter, Write & Exit

This continues until all threads finish.



The output shows:

- Multiple readers entering simultaneously
- Writers waiting if readers are active
- Writers entering only when no readers are present
- Order varies due to thread scheduling

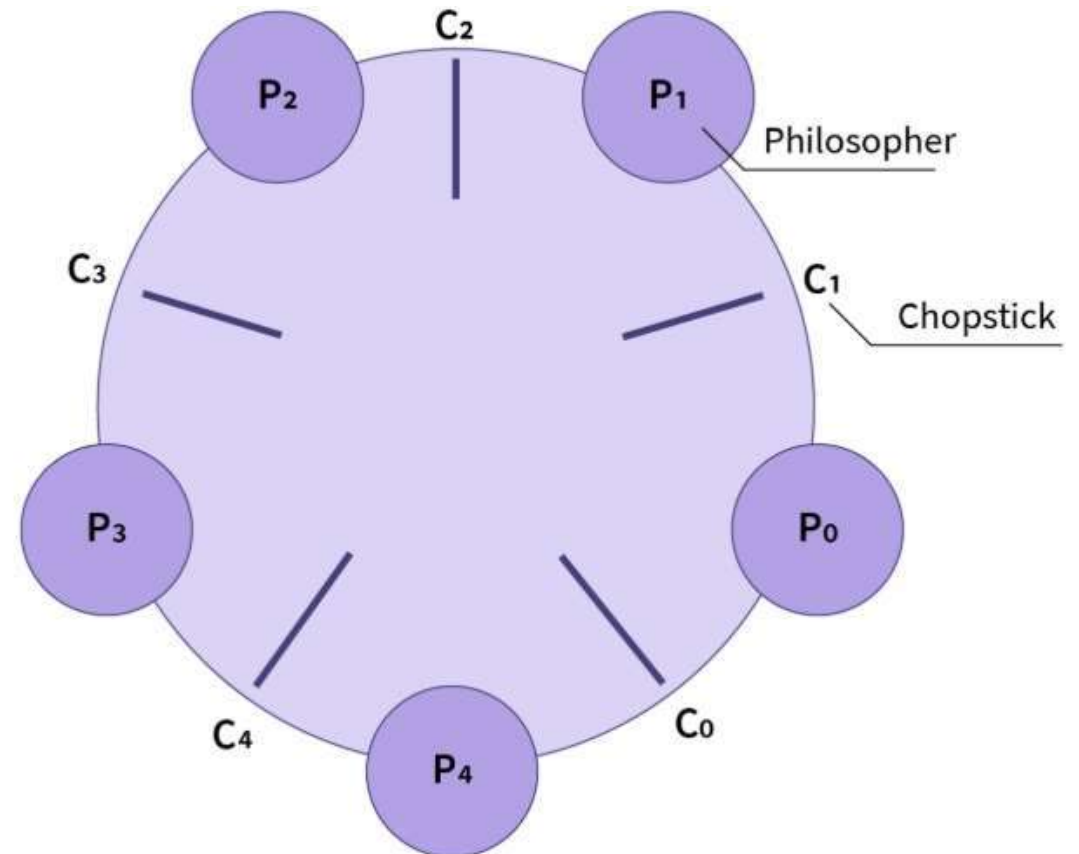
Lab 07: Dining Philosopher's Problem

This is the classic Dining Philosophers Problem, a well-known concurrency challenge in computer science.

It demonstrates process synchronization, where multiple processes **share limited resources and must avoid data inconsistency**

Setup:

- 5 philosophers sit around a circular table
- Each philosopher has a bowl of spaghetti
- One fork between each pair (5 forks total)
- Philosophers numbered 0 to 4 clockwise





Rules:

1. To eat, a philosopher needs both left AND right chopsticks
2. Each 2 chopstick can be used by only one philosopher at a time
3. After eating, both chopsticks must be put down
4. Eating starts only after acquiring both chopsticks

The Challenge:

Design a concurrent algorithm that avoids **deadlock** and **starvation**,
By ensuring all philosophers can eat repeatedly without being blocked forever.

The Core Problem:

- Deadlock:** all philosophers hold one chopstick and wait forever
- Starvation:** some philosophers never get a chance to eat



What the program does

There are **4 philosophers** ($n = 4$).

Each philosopher:

- First tries to pick up the **left fork**
- Then the **right fork**
- Then “eats” (completes dinner)
- Then releases both forks

The loop continues until **all 4 philosophers finish eating**

Output consists of:

- Fork pickup messages
- Waiting messages
- Completion messages
- Fork release messages
- It loops until all 4 philosophers finish eating
- Exact order may vary, but overall pattern remains the same

Lab 08: Deadlock

Deadlock is a situation where **processes are permanently blocked because each is waiting for resources held by others.**

OR

Deadlock is when processes are stuck forever, waiting for each other.

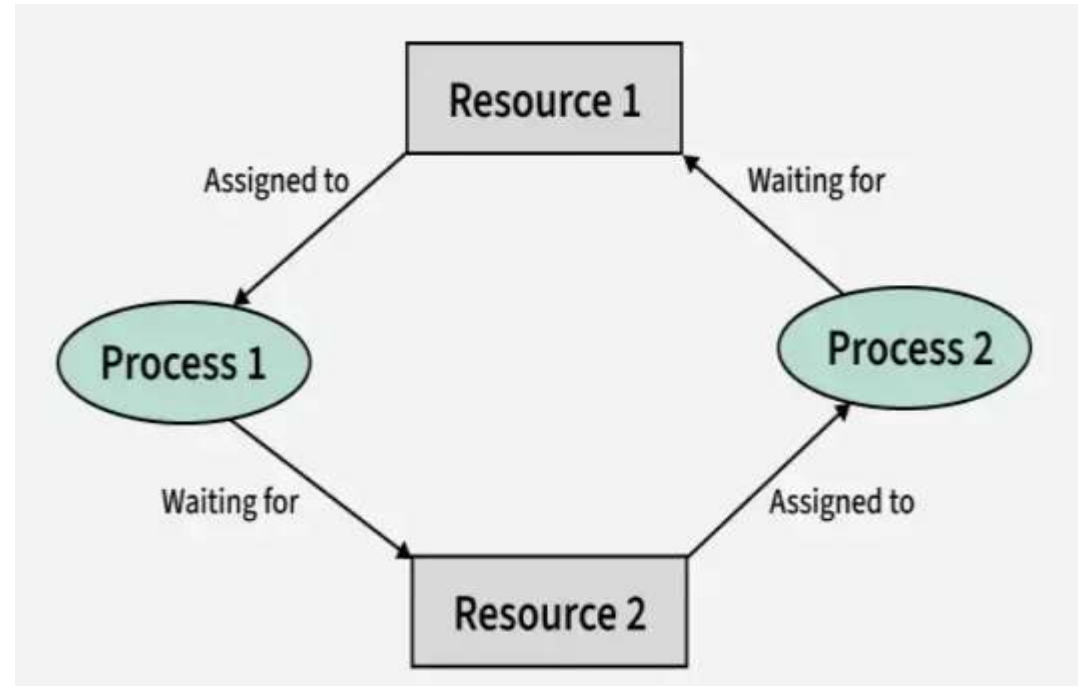
In OS:

Process A is waiting for B

Process B is waiting for A

Example:

- Process P1 holds Resource R1 and requests R2.
 - Process P2 holds Resource R2 and requests R1.
- Neither process can proceed causing a deadlock.**





4 Conditions of Deadlock

- 1. Mutual Exclusion:** Only one process can use a resource at any given time.
i.e. Printer cannot be shared (resources is non-sharable).
- 2. Hold and Wait:** A process is holding at least one resource at a time and is waiting to acquire other resources held by some other process.
i.e. Holding printer, waiting for memory



4 Conditions of Deadlock

3. No Preemption: A resource cannot be taken from a process unless the process releases the resource.

I.e.

Process P1 is printing a document

Process P2 also wants the printer

Can OS stop P1 in the middle and take printer? NO

Because:

Printing is already in progress

Half-printed pages = **corrupted output**

So what happens?

P1 must **finish printing first**

Only then printer is released

Then P2 can use it

So “No Preemption” means:

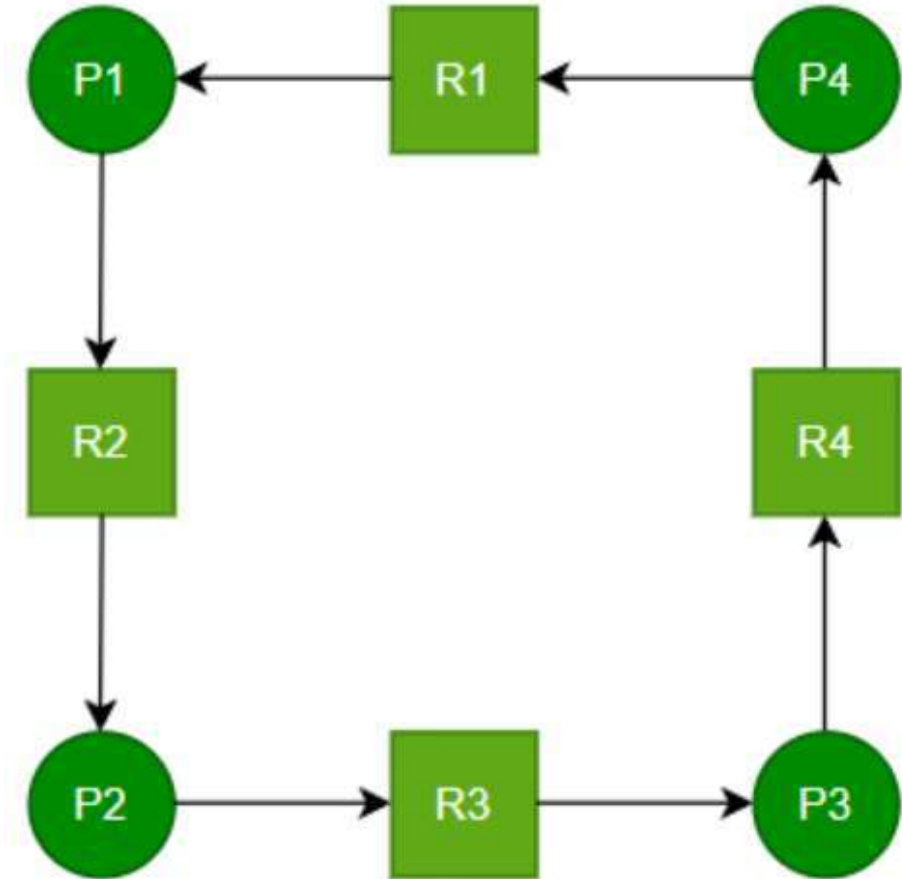
Once a process gets a resource, the OS cannot interrupt or take it back forcefully.

4 Conditions of Deadlock

4. Circular Wait: set of processes are waiting for each other in a circular fashion.

For example, imagine four processes **P1**, **P2**, **P3**, and **P4** and four resources **R1**, **R2**, **R3**, and **R4**.

P1 is holding R1 and waiting for R2 (which is held by P2).
P2 is holding R2 and waiting for R3 (which is held by P3).
P3 is holding R3 and waiting for R4 (which is held by P4).
P4 is holding R4 and waiting for R1 (which is held by P1).





Deadlock Detection

The deadlock Detection Algorithm is of two types:

1. Wait-for-Graph Algorithm
2. Banker's Algorithm

1. Wait-for-Graph Algorithm

A Wait-For Graph shows **which process is waiting for which process.**

Nodes $\rightarrow$ Processes (P1, P2, P3...)

Arrow $\rightarrow$ "waiting for"

If there is a **cycle** $\rightarrow$ **Deadlock exists**

i.e.

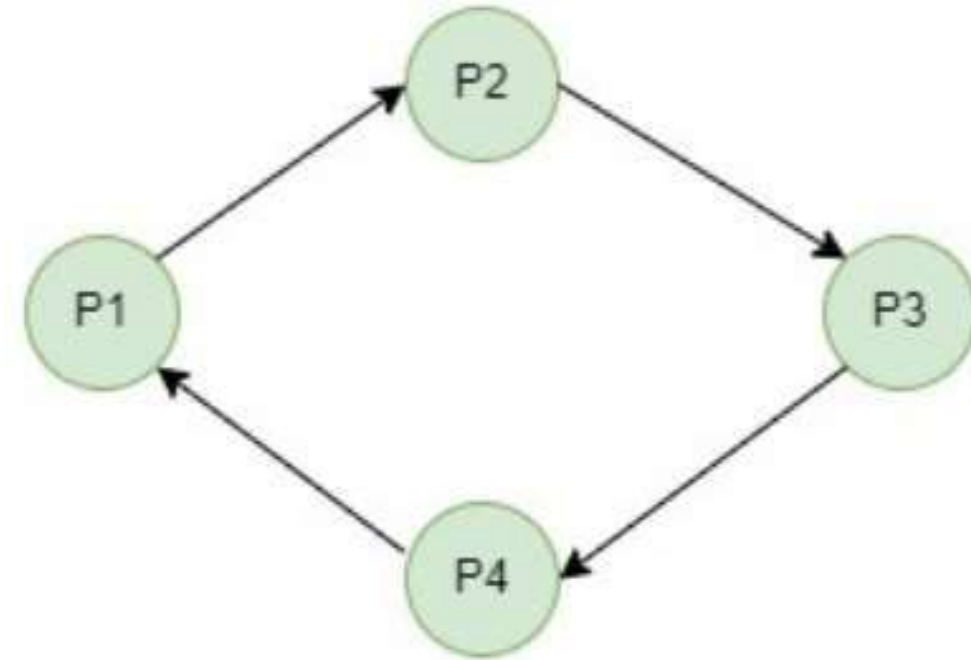
$P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$

-Forms a loop

-All are waiting so, **Deadlock**

$P1 \rightarrow P2 \rightarrow P3$

No cycle so, **No deadlock**



Calculation: CASE 1: SAFE STATE

Given

- **Allocation:** P0 (1,0) P1 (2,1) [P0 currently has category of resources: 1 CPUs, 0 Printers]
- **Max:** P0 (2,2) P1 (3,3)
- **Available** (2,2)

STEP 1: Find NEED

Formula: **Need = Max – Allocation**

$$P0: (2,2) - (1,0) = (1,2)$$

$$P1: (3,3) - (2,1) = (1,2)$$

STEP 2: CHECK + EXECUTION [Compare Need ≤ Available]

$$\text{Available} = (2,2)$$

$$P0: (1,2) \leq (2,2) \checkmark \rightarrow \text{run}$$

$$\text{Available} = (2,2) + (1,0) = (3,2)$$

$$\text{Now Available} = (3,2)$$

$$P1: (1,2) \leq (3,2) \checkmark \rightarrow \text{run}$$

$$\text{Available} = (3,2) + (2,1) = (5,3)$$

STEPS:

1. Calculate → NEED
2. Compare → Need ≤ Available
3. If yes → run process
4. Add allocation back
5. Repeat

RESULT

All finish → SAFE (No deadlock)

Calculation: CASE 2: DEADLOCK

Given

- **Allocation:** P0 (1,0) P1 (0,1) [P0 currently has category of resources: 1 CPUs, 0 Printers]
- **Max:** P0 (2,1) P1 (1,2)
- **Available** (1,0)

STEP 1: Find NEED

Formula: Need = Max – Allocation

$$P0: (2,1) - (1,0) = (1,1)$$

$$P1: (1,2) - (0,1) = (1,1)$$

STEP 2: CHECK + EXECUTION [Compare Need $\leq$ Available]

Available = (1,0)

P0: (1,1) ✗ (1 $\leq$ 1 OK, 1 $\leq$ 0 FAIL)

P1: (1,1) ✗ (1 $\leq$ 1 OK, 1 $\leq$ 0 FAIL)

RESULT

- No process can run
- No one releases resources
- System is stuck
So, DEADLOCK

Lab 09: Banker's Algorithm

Banker's Algorithm is a method used by an operating system to avoid deadlocks by **only giving resources when it is sure that all processes can finish safely.**

It checks before allocating: *"Will everyone still be able to finish?"*

What data it uses:

For each process:

- **Allocation** → resources currently given
- **Max** → maximum resources it may need
- **Need** → what it still needs
 - **Need = Max – Allocation**
- **Available** → free resources in system



Key Concepts in Banker's Algorithm

1. Safe State:

There is **some order** in which all processes can finish.

2. Unsafe State:

No matter what you try, **someone might get stuck later**.

Current Available Resources = 3

Process	Max	Allocation	Need
P0	7	1	6
P1	5	2	3
P2	8	3	5

Step	Available	Process Selected	Condition (Need ≤ Available)	New Available
1	3	P1	$3 \leq 3 \checkmark$	$3 + 2 = 5$
2	5	P2	$5 \leq 5 \checkmark$	$5 + 3 = 8$
3	8	P0	$6 \leq 8 \checkmark$	$8 + 1 = 9$

STEPS:

1. Calculate → NEED
2. Compare → Need ≤ Available
3. If yes → run process
4. Add allocation back
5. Repeat

Safe Sequence

P1 → P2 → P0

Total Available Resources = 3

Process	Allocated	Max	Need (Max - Alloc)
P1	1	7	6
P2	2	5	3
P3	3	8	5



Total Available Resources = 3

Total Allocated = 1 + 2 + 3 = 6

Current Available Resources = Total A.R - Allocated = 3 - 6 = -3

Process	Need	Available	Can Execute?
P1	6	-3	No
P2	3	-3	No
P3	5	-3	No



Unsafe State

Example:

Considering a system with five processes P_0 through P_4 and three resources of type A, B, C., with **current availability**

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Need Matrix

$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$

So, the content of Need Matrix is:

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Safe State and Safe Sequence Analysis

Applying the Safety algorithm on the given system,

Step 1 of Safety Algo

$m = 3, n = 5$

Work = Available

Work =

3	3	2
---	---	---

Finish =

0	1	2	3	4
false	false	false	false	false

Step 2

For $i = 0$

$\text{Need}_0 = 7, 4, 3$

Finish [0] is false and $\text{Need}_0 > \text{Work}$

So P_0 must wait

But $\text{Need} \leq \text{Work}$

Step 2

For $i = 1$

$\text{Need}_1 = 1, 2, 2$

Finish [1] is false and $\text{Need}_1 < \text{Work}$

So P_1 must be kept in safe sequence

Step 3

$3, 3, 2$ $2, 0, 0$
 $Work = Work + Allocation_1$

Work =

A	B	C
5	3	2

Finish =

0	1	2	3	4
false	true	false	false	false

Step 2

For i = 2

$Need_2 = 6, 0, 0$

Finish [2] is false and $Need_2 > Work$
So P_2 must wait



Step 2

For i = 3

$Need_3 = 0, 1, 1$

Finish [3] is false and $Need_3 < Work$
So P_3 must be kept in safe sequence



Step 3

$$\text{Work} = \overset{5, 3, 2}{\text{Work}} + \overset{2, 1, 1}{\text{Allocation}_3}$$

Work =

A	B	C
7	4	3

Finish =

0	1	2	3	4
false	true	false	true	false

Step 2

For i = 4

$\text{Need}_4 = 4, 3, 1$

Finish [4] is false and $\overset{4, 3, 1}{\text{Need}_4} < \overset{7, 4, 3}{\text{Work}}$
So P_4 must be in safe sequence



Step 3

$7, 4, 3$ $0, 0, 2$
 $\text{Work} = \text{Work} + \text{Allocation}_4$

Work =

A	B	C
7	4	5

Finish =

0	1	2	3	4
false	true	false	true	true

Step 2

For $i = 0$

$\text{Need}_0 = 7, 4, 3$

Finish [0] is false and $\text{Need}_0 < \text{Work}$
So P_0 must be in safe sequence



Step 3

$$\text{Work} = \overset{7,4,5}{\text{Work}} + \overset{0,1,0}{\text{Allocation}_0}$$

Work =

A	B	C
7	5	5

Finish =

0	1	2	3	4
true	true	false	true	true

Step 2

For i = 2

$$\text{Need}_2 = 6, 0, 0$$

Finish [2] is false and $\text{Need}_2 < \text{Work}$

So P_2 must be in safe sequence



Step 3

$7, 5, 5$ $3, 0, 2$
Work = Work + Allocation₂

Work =

A	B	C
10	5	7

Finish =

0	1	2	3	4
true	true	true	true	true

Step 4

Finish [i] = true for $0 \leq i \leq n$

Hence the system is in Safe State

The Safe Sequence is P_1, P_3, P_4, P_0, P_2

Lab 10: Disk Scheduling

Disk scheduling algorithms are used by the **Operating System (OS)** to manage **read and write requests** of a computer:

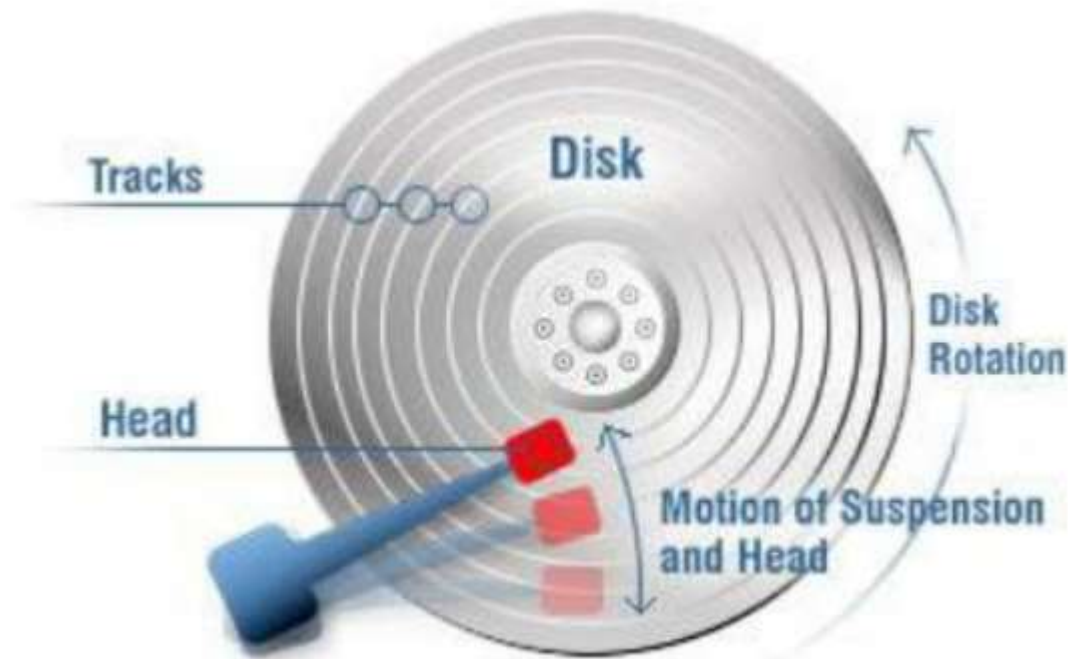
- Hard Disk Drive (HDD)
- Solid State Drive (SSD)

When multiple requests come, the OS decides **which request to serve first**

Basic Disk Component

Tracks

- Circular rings on the disk where data is stored
- Numbered (0, 1, 2, ...)
- **Disk head moves between tracks**





Important Terms

1. Seek Time

Time taken by disk head to move to required track

High seek time = poor performance

2. Total Head Movement

Total Head Movement = $\sum | \text{Current Position} - \text{Next Request} |$

Disk Scheduling Algorithms

1. FCFS (First-Come, First-Served)

Serve requests in the order they arrive

Example

Initial Head = 53

Queue = [98, 183, 37, 122, 14, 124, 65, 67]

Total Head Movement =

$$|53-98| + |98-183| + |183-37| + |37-122| + |122-14| + |14-124| + |124-65| + |65-67|$$

$$= 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2$$

= 640 tracks

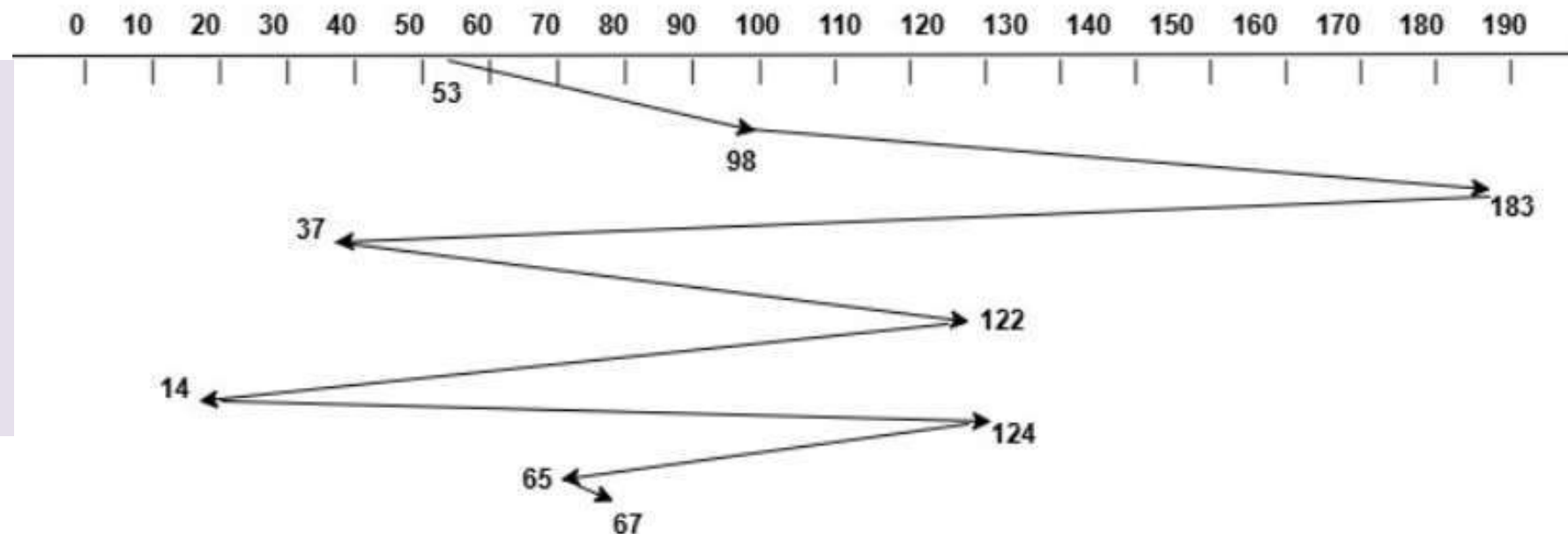
So the total head movement using FCFS disk scheduling algorithm is 640 tracks.

Advantages:

- Simple
- No starvation

Disadvantages:

- Very high head movement
- Poor performance



Disk Scheduling Algorithms

2. SSTF (Shortest Seek Time First)

Select request closest to current head position

Example

Initial Head = 53

Queue = [98, 183, 37, 122, 14, 124, 65, 67]

SSTF ORDER: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Total Head Movement =

$$|53-65| + |65-67| + |67-37| + |37-14| + |14-98| + |98-122| + |122-124| + |124-183|$$

$$= 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59$$

= 236 tracks

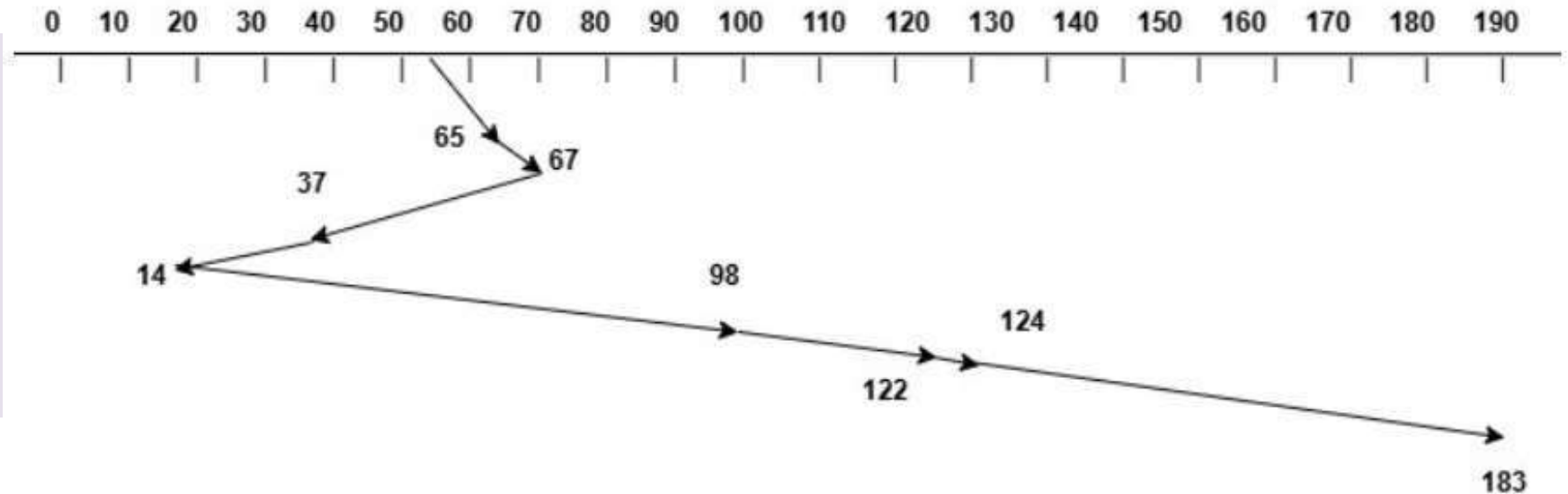
So the total head movement using SSTF disk scheduling algorithm is 236 tracks.

Advantages:

- Much faster than FCFS
- Reduces head movement

Disadvantages:

- Starvation possible (far requests may wait forever)



3. SCAN (Elevator Algorithm)

- Move in one direction until it reaches the end of the disk
- Serve all requests
- Then reverse direction (The head reverses its direction and serves requests in opposite direction)

Example

consider a disk having 200 tracks and requests are arriving in the order: **[82, 170, 43, 140, 34, 190]**.

The initial position of the **disk head is at track 70** and the **head is moving towards the right**.

Advantages:

- Better response time
- No starvation

Disadvantages:

- End tracks may wait longer

Step 1: Move Right

70 → 82 → 140 → 170 → 190 → 199

Step 2: Reverse (Left)

199 → 43 → 34

Total Head Movement =

$$\begin{aligned} &= (199 - 70) + (199 - 34) \\ &= 129 + 165 \\ &= \mathbf{294 \text{ tracks}} \end{aligned}$$

